Árboles en Estructuras de Datos

Lo que entendí, cómo lo entendí y lo que casi me hizo tropezar

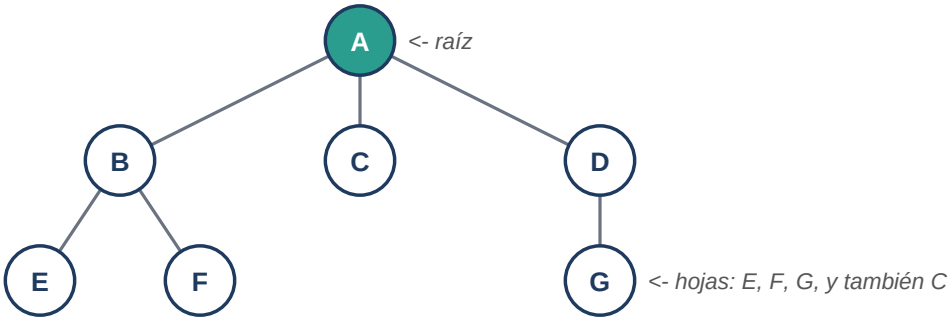
Estos apuntes están escritos desde mi lugar de estudiante: explico cada idea con mis propias palabras, con analogías, ejemplos paso a paso y los errores típicos que conviene evitar. Se basan en la guía de estudio *Árboles generales y árboles binarios*. La idea es que sirvan para repasar antes de un examen sin releer todo.

Qué voy a repasar

- **Qué es un árbol** y cómo se habla de él (raíz, hoja, altura...).
- **Árboles generales** y cómo se guardan en memoria.
- **Árboles binarios**: tipos y propiedades.
- **Recorridos**: preorden, inorden, postorden y por niveles.
- **Árbol binario de búsqueda (ABB)**: buscar, insertar, eliminar y complejidad.
- **Errores frecuentes** y **ejercicios resueltos**.

1. Primero lo primero: qué es un árbol

Mi forma de imaginarlo: **un organigrama de empresa o las carpetas de mi computador**. Hay un elemento arriba (la carpeta principal), del que cuelgan otros, y de esos cuelgan otros más. Un árbol es una estructura **no lineal** que guarda relaciones jerárquicas entre elementos llamados **nodos**, unidos por **aristas**. En una lista hay una sola fila para recorrer todo; en un árbol hay ramas y hay que decidir por cuál ir.



El vocabulario (con mi manera de recordarlo)

Término	Qué significa	Ejemplo en el dibujo
Raíz	El nodo de arriba; no tiene padre.	A
Padre / hijo	Relación directa entre dos nodos conectados.	A es padre de B; B es hijo de A
Hermanos	Nodos con el mismo padre.	B, C y D

Término	Qué significa	Ejemplo en el dibujo
Hoja	Nodo sin hijos (el final de una rama).	C, E, F, G
Nodo interno	Nodo con al menos un hijo.	A, B, D
Camino	Secuencia de nodos conectados.	A - B - E
Profundidad	Aristas desde la **raíz** hasta ese nodo (hacia abajo).	Profundidad de E = 2
Altura	Aristas del camino más largo desde un nodo hasta una hoja. La altura del árbol se mide desde la raíz.	Altura del árbol = 2
Subárbol	Un nodo con todos sus descendientes.	B, E, F forman un subárbol

Detalle en el que me fijé
En el dibujo de la guía el texto dice que las hojas son E, F y G, pero **C también es hoja** porque no tiene hijos. Me sirvió para recordar la regla: *hoja = sin hijos*, sin importar en qué nivel esté.

Truco para no confundirme
Profundidad **baja** desde la raíz (empieza en 0 arriba); altura **sube** desde las hojas (las hojas tienen altura 0). Y una regla muy útil: un árbol conectado y sin ciclos con **n** nodos tiene siempre **n - 1** aristas (cada nodo, salvo la raíz, tiene exactamente una arista hacia su padre).

2. Árboles generales

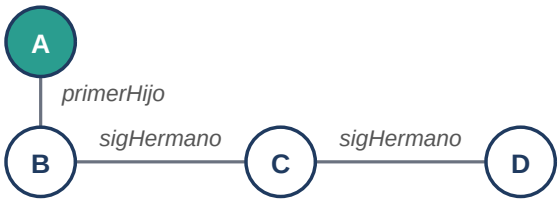
Son los más libres: cada nodo puede tener **cero, uno o muchos hijos**, sin límite fijo. Aparecen en sistemas de archivos (carpetas y subcarpetas), organigramas, menús, categorías y árboles de sintaxis.

Cómo se guardan en memoria

Aquí me costó un poco, porque no se sabe de antemano cuántos hijos tendrá cada nodo. Hay dos soluciones comunes:

- **Lista de hijos:** cada nodo guarda una colección (por ejemplo `vector<Nodo*> hijos`) con los punteros a sus hijos. Es flexible, pero hay que administrar esa colección.
- **Primer hijo - siguiente hermano:** cada nodo guarda solo **dos** punteros: uno a su primer hijo y otro a su siguiente hermano. Así cualquier árbol general cabe en una estructura de dos enlaces.

```
struct Nodo {
    int dato;
    Nodo* primerHijo;
    Nodo* siguienteHermano;
};
```

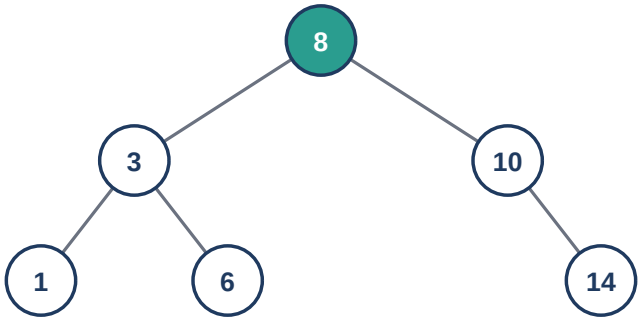


A tiene tres hijos (B, C, D), pero solo guarda a B.
B enlaza con C y C con D.

Ventaja	Desventaja
Siempre dos punteros por nodo, sin importar cuántos hijos tenga.	Para llegar al hijo número k hay que ir saltando de hermano en hermano, y el esquema es menos intuitivo.

3. Árboles binarios

Un árbol binario es un árbol donde **cada nodo tiene como máximo dos hijos**, distinguidos como **izquierdo** y **derecho**. Ojo: que un hijo falte es válido, y la posición importa (un único hijo a la izquierda no es lo mismo que uno a la derecha). Este es el árbol que uso como ejemplo durante todos estos apuntes:



Representación enlazada en C++

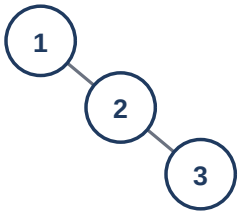
```
struct Nodo {
    int dato;
    Nodo* izq;
    Nodo* der;
    Nodo(int valor) : dato(valor), izq(nullptr), der(nullptr) {}
};
```

Tipos de árboles binarios

Tipo	Idea en una frase	Mi forma de recordarlo
Estricto o lleno	Cada nodo tiene 0 o 2 hijos.	Nadie tiene "hijo único".
Perfecto	Todos los niveles completos; todas las hojas a la misma profundidad.	Un triángulo perfectamente relleno.
Completo	Todos los niveles llenos salvo quizá el último, que se llena de izquierda a derecha.	Se llena como una fila de cine: sin huecos a la izquierda.

Tipo	Idea en una frase	Mi forma de recordarlo
Degenerado	Cada nodo interno tiene un solo hijo.	Es una lista disfrazada de árbol.

Ejemplo de árbol degenerado (inserciones 1, 2, 3 en un ABB):



Comprobando propiedades con mi ejemplo
Con altura **h** (en aristas), un árbol binario tiene como máximo $\mathbf{2^{h+1} - 1}$ nodos. Comprobación con nuestro ejemplo: su altura es 2, así que el máximo seria 2^3 - 1 = 7 nodos y tiene 6. Por eso **no es perfecto**. Tampoco es estricto (el 10 solo tiene un hijo) ni completo (al nivel de 1, 6 y 14 le falta el hijo izquierdo del 10 antes de llegar al 14).

4. Recorridos: visitar todos los nodos con orden

Recorrer es visitar los nodos siguiendo un orden definido. Los tres recorridos **en profundidad** se distinguen solo por **cuándo se visita la raíz** respecto a sus dos subárboles. Eso lo resumo así:

Recorrido	Orden	Pista para recordarlo	Resultado en mi ejemplo
Preorden	raíz, izquierda, derecha	la raíz va **pre** (antes)	8, 3, 1, 6, 10, 14
Inorden	izquierda, raíz, derecha	la raíz va **in** (en medio)	1, 3, 6, 8, 10, 14
Postorden	izquierda, derecha, raíz	la raíz va **post** (después)	1, 6, 3, 14, 10, 8
Por niveles	nivel 0, nivel 1, nivel 2...	piso por piso, de izq. a der.	8, 3, 10, 1, 6, 14

Cómo hago el recorrido a mano (ejemplo con preorden)

- Visito el **8** (raíz) y lo anoto. Bajo a su subárbol izquierdo.
- Visito el **3**. Bajo a su izquierda: el **1**, que es hoja; lo anoto y subo.
- Vuelvo al 3 y voy a su derecha: el **6**. Termina el subárbol izquierdo del 8.
- Ahora el subárbol derecho del 8: visito el **10**; no tiene hijo izquierdo; sigo con su derecho, el **14**.
- Resultado: 8, 3, 1, 6, 10, 14.

Implementación recursiva

La clave: **el caso base es el puntero nulo**. Si llego a un `nullptr`, no hay nada que visitar y regreso. Las tres funciones son casi idénticas; solo cambia la línea del `cout` de lugar.

```
void preorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    cout << raiz->dato << ' ';   // visito la raíz primero
    preorden(raiz->izq);
    preorden(raiz->der);
}
void inorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    inorden(raiz->izq);
    cout << raiz->dato << ' ';   // la raíz en medio
    inorden(raiz->der);
}
void postorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    postorden(raiz->izq);
    postorden(raiz->der);
    cout << raiz->dato << ' ';   // la raíz al final
}
```

Recorrido por niveles (BFS)

Aquí no sirve la recursión directa: se usa una **cola**. Saco un nodo, lo visito y meto a sus hijos al final de la cola; así los de un nivel salen antes que los del siguiente.

```
#include <queue>
void porNiveles(Nodo* raiz) {
    if (raiz == nullptr) return;
    queue<Nodo*> q;
    q.push(raiz);
    while (!q.empty()) {
        Nodo* n = q.front(); q.pop();
        cout << n->dato << ' ';
        if (n->izq) q.push(n->izq);
        if (n->der) q.push(n->der);
    }
}
```

5. Árbol binario de búsqueda (ABB)

Un ABB es un árbol binario con una **regla de orden**: para cada nodo, las claves del subárbol izquierdo son **menores** y las del derecho **mayores** (según la convención adoptada). Los duplicados necesitan una regla explícita (por ejemplo, no permitirlos). **Todo ABB es un árbol binario, pero no todo árbol binario es un ABB.**

> **Dato que me ahorro memorizar**
> Si hago el recorrido **inorden** de un ABB, obtengo las claves **ordenadas de menor a mayor**. En nuestro ejemplo: 1, 3, 6, 8, 10, 14. Es una buena forma de comprobar si un árbol es realmente un ABB.

Insertar paso a paso: 8, 3, 10, 1, 6, 14

Clave	Comparaciones	Dónde queda
8	El árbol está vacío	Es la raíz
3	3 < 8: izquierda	Hijo izquierdo de 8
10	10 > 8: derecha	Hijo derecho de 8
1	1 < 8 (izq.), 1 < 3 (izq.)	Hijo izquierdo de 3
6	6 < 8 (izq.), 6 > 3 (der.)	Hijo derecho de 3
14	14 > 8 (der.), 14 > 10 (der.)	Hijo derecho de 10

El resultado es justamente el árbol de la sección 3, y su recorrido inorden da 1, 3, 6, 8, 10, 14.

Las tres operaciones

- **Buscar:** comparo la clave con el nodo actual y bajo a izquierda o derecha. Ejemplo, buscar 6: 8 (menor, voy a la izquierda), 3 (mayor, voy a la derecha), 6: encontrado en 3 comparaciones.
- **Insertar:** recorro como en buscar hasta hallar una posición vacía y ahí coloco el nuevo nodo.
- **Eliminar:** tiene tres casos, según el nodo a eliminar.

Caso	Qué hago	Ejemplo en mi árbol
Hoja	Simplemente la quito.	Eliminar 1
Un hijo	El hijo ocupa el lugar del nodo eliminado.	Eliminar 10: el 14 sube a su lugar
Dos hijos	Reemplazo por el sucesor inorden (el menor del subárbol derecho) o por el predecesor (el mayor del izquierdo), y luego elimino ese nodo de su sitio original.	Eliminar 3: sucesor = 6. El 6 ocupa el lugar del 3

```
Nodo* insertar(Nodo* raiz, int valor) {
    if (raiz == nullptr) return new Nodo(valor);
    if (valor < raiz->dato)      raiz->izq = insertar(raiz->izq, valor);
    else if (valor > raiz->dato) raiz->der = insertar(raiz->der, valor);
    // si es igual: no hago nada (regla explícita: sin duplicados)
    return raiz;
}
bool buscar(Nodo* raiz, int valor) {
    if (raiz == nullptr) return false;
    if (valor == raiz->dato) return true;
    return valor < raiz->dato ? buscar(raiz->izq, valor)
                              : buscar(raiz->der, valor);
}
void liberar(Nodo* raiz) {       // libero memoria: es un postorden
    if (raiz == nullptr) return;
    liberar(raiz->izq);
    liberar(raiz->der);
    delete raiz;
}
```

Nota: `liberar` es un postorden a propósito: primero hay que liberar a los hijos y recién después al padre; si no, se perderían los punteros.

Complejidad: por que a veces es O(log n) y a veces O(n)

Operación	Árbol equilibrado	Peor caso (sesgado)
Buscar	O(log n)	O(n)
Insertar	O(log n)	O(n)
Eliminar	O(log n)	O(n)
Recorrido completo	O(n)	O(n)

La idea: en cada comparación descarto una rama completa. Si el árbol está equilibrado, la altura es cercana a $\log_2(n)$ y hacen falta pocos pasos. Pero si inserto 1, 2, 3, ..., n **ya ordenados**, cada nodo queda a la derecha del anterior, el árbol se vuelve una lista (degenerado) con altura n - 1, y buscar puede recorrer los n nodos. Para evitarlo existen los árboles autoequilibrados, como **AVL** y **Red-Black**.

6. Árbol general vs. árbol binario

Característica	Árbol general	Árbol binario
Máximo de hijos	Sin límite fijo	2
Orden de los hijos	Según el modelo	Izquierdo y derecho
Ejemplos	Directorios, jerarquías	ABB, árboles de expresion, heaps
Recorrido	Depende de la representación	Pre/in/postorden y por niveles

7. Errores frecuentes y buenas practicas

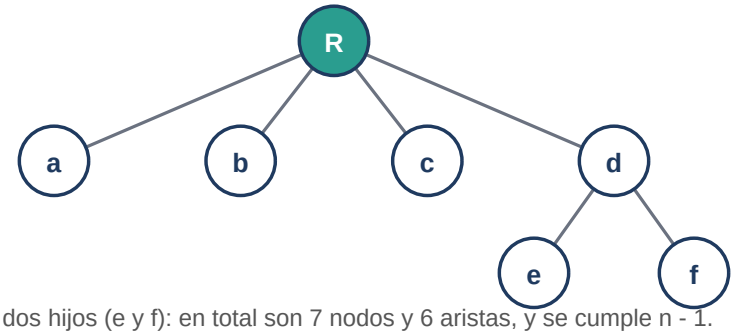
- **Olvidar el caso base** (`nullptr`): provoca fallos al llegar al final de una rama.
- **Confundir árbol binario con ABB**: el ABB tiene además la condición de orden.

- **Dar por hecho que un ABB está equilibrado**: depende del orden de inserción.
- **Dejar los nodos sin liberar** en C++: hay que definir quién es responsable de liberarlos para evitar fugas de memoria.
- **No decidir que pasa con los valores repetidos**: elegir una regla y mantenerla.
- **No probar casos límite**: árbol vacío, árbol de un solo nodo y árbol con varias ramas.

8. Ejercicios de repaso resueltos

1. ¿En un árbol con 15 nodos, cuántas aristas hay?
n - 1 = **14 aristas**.

2. Un árbol general donde la raíz tenga cuatro hijos y uno de ellos tenga dos hijos.



Elegí que el hijo **d** tuviera dos hijos (e y f): en total son 7 nodos y 6 aristas, y se cumple n - 1.

3. Recorridos del árbol de la sección 3.
Preorden: 8, 3, 1, 6, 10, 14. Inorden: 1, 3, 6, 8, 10, 14. Postorden: 1, 6, 3, 14, 10, 8.

4. Insertar 8, 3, 10, 1, 6, 14 en un ABB: ¿cuál es el recorrido inorden?
1, 3, 6, 8, 10, 14 (siempre ordenado en un ABB).

5. ¿Por qué buscar en un ABB puede llegar a O(n)?
Porque el árbol puede quedar sesgado (por ejemplo, si se insertan claves ya ordenadas) y su altura llegar a n - 1, con lo que se debe recorrer casi todos los nodos.

6. Ventaja y desventaja de primer hijo - siguiente hermano.
Ventaja: solo dos punteros por nodo sin importar el número de hijos. Desventaja: acceder al hijo k exige recorrer la cadena de hermanos, y es menos directo que una lista de hijos.

Resumen en tres líneas
Los árboles modelan relaciones jerárquicas. Los **generales** permiten cualquier cantidad de hijos; los **binarios** limitan cada nodo a dos. Los **recorridos** son técnicas para visitar todos los nodos (la recursión y el caso base nulo son la clave). Los **ABB** aprovechan el orden para buscar, insertar y eliminar con eficiencia, siempre que su altura se mantenga controlada.